Priority, Round Robin, Multilevel Queues, and Multiprocessor Scheduling

Lecture Notes — Lecture 9

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 29, 2026

Concepts That Click

9.1 Beyond FCFS and SJF

Reading: Silberschatz et al., Ch. 5 §5.3, pp. 192–199 and §5.5 (p. 200 onward) [2]; Stallings, Ch. 9 §9.2 and Ch. 10 §10.1 [3].

A laptop runs a video call, a compiler, and a backup at the same time. Under FCFS the backup can block the call for as long as its CPU burst lasts; under SJF the scheduler would need to know burst lengths that nobody knows in advance. Lecture 8 left us exactly there: FCFS is simple but suffers the **convoy effect**, and SJF is optimal for average waiting time but needs the next burst τ. This chapter asks how real schedulers decide which process runs next — and how they stay fair while doing it.

Four ideas answer the question, and real kernels combine them: **Priority** scheduling (some processes simply matter more), **RR** or Round Robin (everyone gets a turn quickly), the **MLQ** or multilevel queue (processes come in classes), and the **MLFQ** or multilevel feedback queue (the scheduler *learns* which processes are short). A fifth question appears once there is more than one CPU: which CPU, and which queue?

To make every trade-off a number rather than an adjective, one five-process workload, W, is scheduled by every policy in the chapter (Table 9.1). Times are in milliseconds. Throughout, a **lower priority number means higher priority** (Silberschatz's convention), and ties go to the earlier arrival. The total work is $8 + 4 + 9 + 5 + 2 = 28$ ms, so every schedule of W without idle time ends at 28.

Process	Arrival (ms)	Burst τ (ms)	Priority	Character
P1	0	8	3	medium batch job
P2	1	4	1	interactive, important
P3	2	9	4	long compute job
P4	3	5	2	system daemon
P5	4	2	5	short, low-importance

Table 9.1: The running workload W.

The three metrics, per process

The metrics are those of Lecture 8, computed per process from a Gantt chart: **TAT** (turnaround) = completion − arrival; **WT** (waiting) = TAT − τ; **RT** (response) = first dispatch − arrival.

Example 9.1 (FCFS baseline on W)**.** *FCFS runs the processes in arrival order (Fig. 9.1). The completion times are 8, 12, 21, 26, 28, so TAT* $= 8, 11, 19, 23, 24$ *(sum 85, average 17.0 ms) and WT* $= 0, 7, 10, 18, 22$ *(sum 57, average* $57/5 = 11.4$ *ms). Because no process is preempted, RT equals WT, average 11.4 ms. Note* `P5`*: it needs only 2 ms but waits 22 ms — the convoy behind* `P3`*.*

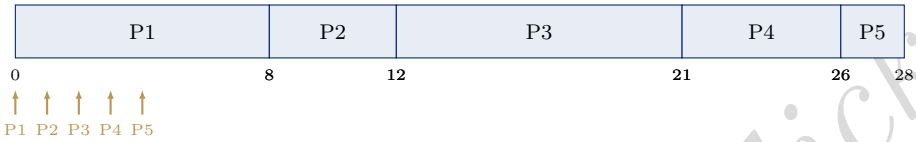


Figure 9.1: FCFS on W. Gold arrows below the axis mark arrivals.

9.2 Priority Scheduling

Reading: Silberschatz et al., Ch. 5 §5.3, p. 192.

SJF already ranked processes by one number, the predicted burst. Priority scheduling generalizes it: rank processes by any number we choose.

Definition (Priority scheduling). *Priority scheduling associates an integer priority with each process and allocates the CPU to the ready process with the highest priority; equal priorities are broken FCFS.*

Priorities may be **internal** — measured quantities such as time limits, memory needs, or the ratio of I/O burst to CPU burst — or **external**, set outside the OS by importance, payment, or department. SJF is the special case in which the priority is the (predicted) next CPU burst: the shorter the burst, the higher the priority. Priority scheduling comes in two forms. In the **non-preemptive** form a newly arrived higher-priority process waits until the running process finishes its burst; in the **preemptive** form it takes the CPU immediately.

Example 9.2 (Non-preemptive priority on W)**.** *At* $t = 0$ *only* `P1` *is ready, so it runs its whole 8 ms burst. At* $t = 8$ *the ready set is {P2(1), P3(4), P4(2), P5(5)}, with priorities in parentheses, so* `P2` *runs to 12. At 12 the best remaining is* `P4` *(priority 2), which runs to 17; then* `P3` *(4) runs to 26; and* `P5` *(5) runs last, to 28 (Fig. 9.2).*

	P1	*P2*	*P3*	*P4*	*P5*	*Average*
Completion	*8*	*12*	*26*	*17*	*28*	
TAT	*8*	*11*	*24*	*14*	*24*	*16.2*
WT	*0*	*7*	*15*	*9*	*22*	*10.6*

For instance, `P3`*:* $TAT = 26 - 2 = 24$, $WT = 24 - 9 = 15$. *The average WT is* $53/5 = 10.6$ *ms.*

Example 9.3 (Preemptive priority on W)**.** *Step by step (Fig. 9.3):*

1. $t = 0$*:* `P1` *runs.*

Concepts That Click

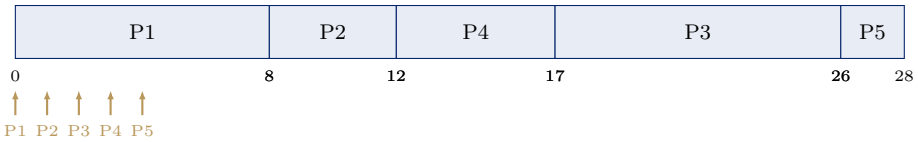


Figure 9.2: Non-preemptive priority on W.

2. $t = 1$: `P2` *(priority 1) arrives and* ***preempts*** `P1` *(priority 3), which keeps 7 ms of work.*
3. $t = 2, 3, 4$: `P3` *(4),* `P4` *(2),* `P5` *(5) arrive; none beats* `P2`*, which finishes at 5.*
4. $t = 5$: *ready* $= \{$*P1(3), P3(4), P4(2), P5(5)*$\}$; `P4` *(2) beats* `P1` *(3) and runs to 10.*
5. $t = 10$: `P1` *resumes and finishes at 17; then* `P3` *runs to 26 and* `P5` *to 28.*

	P1	*P2*	*P3*	*P4*	*P5*	*Average*
Completion	*17*	*5*	*26*	*10*	*28*	
TAT	*17*	*4*	*24*	*7*	*24*	*15.2*
WT	*9*	*0*	*15*	*2*	*22*	*9.6*

The average WT is $48/5 = 9.6$ *ms; the average RT is* $(0 + 0 + 15 + 2 + 22)/5 = 7.8$ *ms.*

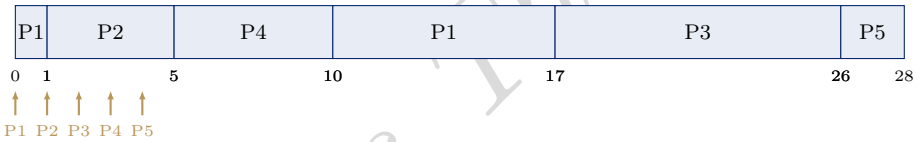


Figure 9.3: Preemptive priority on W.

What priority bought, and who paid

Comparing the policies so far: FCFS gives average WT 11.4 ms (`P2` waits 7, `P5` waits 22); non-preemptive priority gives 10.6 ms (`P2` still 7, `P5` 22); preemptive priority gives 9.6 ms, with the important process `P2` now waiting 0 ms. Non-preemptive priority could not help `P2`, because `P1` already held the CPU when `P2` arrived. But `P5` still waits 22 ms in every case — and with a steady stream of higher-priority arrivals it would wait forever.

Notice also that in preemptive priority `P1` starts first yet finishes after `P2` and `P4`, waiting 9 ms. This is not unfair by the policy's own standard: priority judges *importance*, not length, and two more important processes arrived. SRTF would have made the same preemption at $t = 1$ for a different reason — `P2`'s 4 ms burst beats `P1`'s remaining 7 ms. Same mechanism, different key: that is why SJF is a priority algorithm.

Starvation and aging

Definition (Starvation). *Starvation (indefinite blocking) is the situation of a ready process that never gets the CPU because higher-priority work keeps arriving.*

Definition (Aging). *Aging gradually raises the priority of a process the longer it waits.*

Concepts That Click

Example 9.4 (Bounding the wait with aging)**.** *Priorities run from 0 (highest) to 63, and aging lowers a waiting process's priority number by 1 every 10 ms. In the worst case a process enters at 63; after* $63 \times 10 = 630$ *ms of waiting its priority number is 0, and nothing can outrank it. Its wait is therefore bounded.*

9.3 Round Robin

Reading: Silberschatz et al., Ch. 5 §5.3, p. 194; Stallings, Ch. 9 §9.2, p. 410.

Priority fixes importance but not responsiveness. Time-sharing needs a different promise: everyone gets the CPU soon, for a little while.

Definition (Round Robin). *Round Robin (RR) is FCFS with preemption by a timer: each process runs for at most one time quantum q; if its burst is not finished, it is preempted and placed at the tail of the ready queue.*

The ready queue is a circular FIFO queue, and the timer interrupt of Lecture 2 enforces the quantum. A burst shorter than q releases the CPU voluntarily. One tie rule matters for hand traces, and it is used throughout this chapter: *a process arriving at the instant a quantum expires joins the queue before the preempted process.*

Example 9.5 (RR with $q = 3$ ms on W)**.** *The queue is tracked at each dispatch (Fig. 9.4):*

1. $t = 0$*: only* `P1`*; it runs 0–3.*
2. $t = 3$*:* `P2`*,* `P3`*,* `P4` *arrived during* `P1`*'s slice, so the queue is [P2, P3, P4]; then* `P1` *(5 ms left) rejoins: [P2, P3, P4, P1].* `P2` *runs 3–6.*
3. $t = 6$*:* `P5` *arrived at 4, so it precedes the preempted* `P2` *(1 ms left): [P3, P4, P1, P5, P2].* `P3` *runs 6–9 (6 left), then* `P4` *runs 9–12 (2 left), then* `P1` *runs 12–15 (2 left).*
4. $t = 15$*: the queue is [P5, P2, P3, P4, P1].* `P5` *(2 ms) runs 15–17 and* `P2` *(1 ms left) runs 17–18; both finish without using a full quantum.* `P3` *runs 18–21 (3 left).*
5. $t = 21$*: [P4, P1, P3] — the remainders drain:* `P4` *to 23,* `P1` *to 25,* `P3` *to 28.*

	P1	*P2*	*P3*	*P4*	*P5*	*Average*
First dispatch	*0*	*3*	*6*	*9*	*15*	
Completion	*25*	*18*	*28*	*23*	*17*	
TAT	*25*	*17*	*26*	*20*	*13*	*20.2*
WT	*17*	*13*	*17*	*15*	*11*	*14.6*
RT	*0*	*2*	*4*	*6*	*11*	*4.6*

Average RT falls from 11.4 ms (FCFS) to 4.6 ms, but average WT rises *from 11.4 to 14.6 ms: long jobs are chopped up and wait repeatedly. RR trades turnaround for responsiveness, by design.*

How the quantum changes everything

Table 9.2 repeats the computation for other quanta; context switches are counted as changes of running process between consecutive slices.

Response time grows steadily with q, but average WT does **not** move monotonically: $q = 2$ and $q = 4$ both give 13.0 ms, while $q = 3$ gives 14.6 ms. WT depends on how the quantum boundaries

Concepts That Click

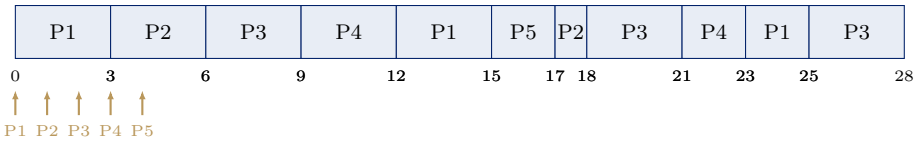


Figure 9.4: RR with $q = 3$ ms on W.

RR on W	avg WT	avg TAT	avg RT	context switches
$q = 1$	13.0	18.6	1.2	25
$q = 2$	13.0	18.6	2.8	13
$q = 3$	14.6	20.2	4.6	10
$q = 4$	13.0	18.6	6.0	8
$q = 5$	12.6	18.2	7.4	6
$q \geq 9$	11.4	17.0	11.4	4

Table 9.2: RR on W for several quanta.

line up with the bursts (8, 4, 9, 5, 2). With $q = 4$, `P2`'s 4 ms burst finishes in one slice; with $q = 3$ it needs two slices and waits a whole round in between. Only the limit is certain: once q is at least the longest burst (9 ms), no process is ever preempted and RR *is* FCFS.

The cost of a small quantum

Every preemption costs a context switch of s ms of pure overhead: save one PCB, load another, and lose warm cache and TLB contents. While bursts outlast the quantum, each q ms of useful work is accompanied by s ms of switching, so the fraction of CPU time doing useful work is

$$\frac{q}{q+s}.$$

Example 9.6 (Switch overhead with $s = 0.5$ ms)**.** *For $q = 1$: $1/1.5 = 66.7\%$ useful. For $q = 3$: $3/3.5 = 85.7\%$. For $q = 10$: $10/10.5 = 95.2\%$. On W, $q = 1$'s 25 switches add $25 \times 0.5 = 12.5$ ms to a 28 ms schedule, while $q = 3$'s 10 switches add $10 \times 0.5 = 5$ ms.*

The rule of thumb follows: make q large compared with the switch time, yet small enough that most CPU bursts (about 80%) finish within one quantum.

RR's guarantee: bounded response

With n processes in the ready queue and quantum q, no process waits more than $(n-1)\,q$ for its next quantum (ignoring switch time), because each of the other $n-1$ processes runs at most q before the queue comes back around. On W with $q = 3$ at most 5 processes are ready, so every process is back on the CPU within $4 \times 3 = 12$ ms; indeed the largest RT is `P5`'s 11 ms ≤ 12. This bound — not the average WT — is why RR is the basis of time-sharing schedulers.

9.4 Multilevel Queue Scheduling

Reading: Silberschatz et al., Ch. 5 §5.3, p. 196 (Fig. 5.6).

Concepts That Click

One policy for every process is a compromise. Real systems split the ready queue by class and give each class its own policy.

Definition (Multilevel queue). *A multilevel queue (MLQ) scheduler partitions the ready queue into several queues; each process is permanently assigned to one queue by its type (memory size, priority, process type), and each queue has its own scheduling algorithm.*

Figure 9.5 shows a four-level example: system processes, interactive processes, and interactive editing are scheduled RR, batch processes FCFS, in decreasing priority.

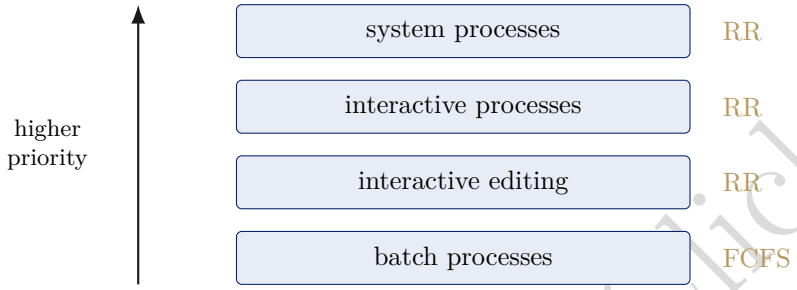


Figure 9.5: A multilevel queue — after Silberschatz et al., Ch. 5 §5.3, p. 196 (Fig. 5.6) [2].

A policy *between* the queues is still needed, and there are two standard choices. Under **fixed priority (preemptive)**, a lower queue runs only when every higher queue is empty, and a new arrival in a higher queue preempts a lower-queue process; lower queues can therefore starve. Under **time slicing between queues**, each queue receives a fixed share of CPU time — for example 80% to the foreground (RR) queue and 20% to the background (FCFS) queue — and no queue starves. MLQ has low overhead, because the class decides the queue once, at creation; it is inflexible for the same reason, since a process that changes behaviour cannot change queue.

Example 9.7 (Two-queue MLQ)**.** *Four processes:* `A` *(arrival 0,* $\tau = 4$*, foreground),* `B` *(1, 6, background),* `D` *(2, 3, background),* `C` *(6, 3, foreground). The foreground queue is RR with* $q = 2$*, the background queue is FCFS, and the queues are scheduled by fixed, preemptive priority (Fig. 9.6).*

1. $t = 0$*–4:* `A` *is alone in the foreground; its two quanta run back to back and it finishes at 4.*
2. $t = 4$*: the foreground is empty, so background* `B` *(the earlier of* `B`*,* `D`*) starts.*
3. $t = 6$*:* `C` *arrives in the foreground and* ***preempts*** `B`*, which has 4 ms left.* `C` *runs 6–9.*
4. $t = 9$*:* `B` *resumes, finishing at 13; then* `D` *runs 13–16.*

WT: `A` $= 0$*,* `C` $= 0$*,* `B` $= (13 - 1) - 6 = 6$*,* `D` $= (16 - 2) - 3 = 11$*; average* $17/4 = 4.25$ *ms. The foreground processes never wait here.*

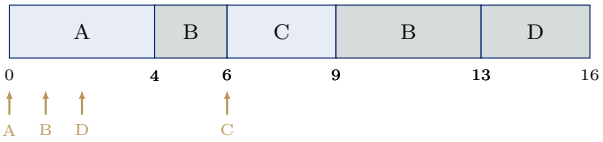


Figure 9.6: Two-queue MLQ (foreground light, background shaded).

Concepts That Click

9.5 Multilevel Feedback Queue Scheduling

Reading: Silberschatz et al., Ch. 5 §5.3, p. 197; Stallings, Ch. 9 §9.2.

MLQ trusts a label fixed at creation. A feedback queue watches what each process actually does, and moves it.

Definition (Multilevel feedback queue). *A multilevel feedback queue (MLFQ) scheduler lets processes move between queues: a process that uses its whole quantum is demoted to a lower queue, so I/O-bound and interactive processes stay in the higher queues; a process that waits too long in a low queue may be promoted (aging), which prevents starvation.*

An MLFQ is defined by five parameters: (1) the number of queues; (2) the scheduling algorithm for each queue; (3) the rule for demoting a process to a lower queue; (4) the rule for promoting a process to a higher queue; (5) the rule for choosing the entry queue of a new process. It is the most general scheme in this chapter: with suitable parameters it behaves like RR, like FCFS, or like an approximation of SJF — without being told any burst length. The configuration used below is shown in Fig. 9.7: Q_0 is RR with $q = 2$ ms, Q_1 is RR with $q = 4$ ms, Q_2 is FCFS, new arrivals enter Q_0, and higher queues preempt lower ones.

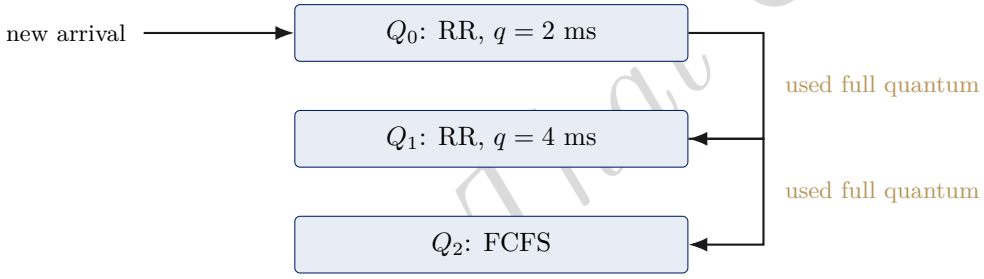


Figure 9.7: A three-level MLFQ — after Silberschatz et al., Ch. 5 §5.3, p. 197 [2].

Example 9.8 (MLFQ on W)**.** *In Fig. 9.8 light bars ran in Q_0, gold in Q_1, grey in Q_2.*

1. *0–10: every process gets 2 ms in Q_0, in arrival order. Only* `P5` *($\tau = 2$) finishes there, at 10. The other four used a full quantum and are demoted to Q_1 with remaining work P1 6, P2 2, P3 7, P4 3.*
2. *10–23: Q_1 runs in order P1 (10–14, 2 left, demoted), P2 (14–16, finishes), P3 (16–20, 3 left, demoted), P4 (20–23, finishes).*
3. *23–28: Q_2 runs* `P1` *(2 ms, to 25) then* `P3` *(3 ms, to 28), FCFS.*

	P1	*P2*	*P3*	*P4*	*P5*	*Average*
Completion	*25*	*16*	*28*	*23*	*10*	
TAT	*25*	*15*	*26*	*20*	*6*	*18.4*
WT	*17*	*11*	*17*	*15*	*4*	*12.8*
RT	*0*	*1*	*2*	*3*	*4*	*2.0*

The shortest job `P5` *finishes at 10 instead of 28 under FCFS — MLFQ discovered it was short without being told. Average RT is 2.0 ms against RR $q = 3$'s 4.6 ms, with the same number of context switches (10).*

Concepts That Click

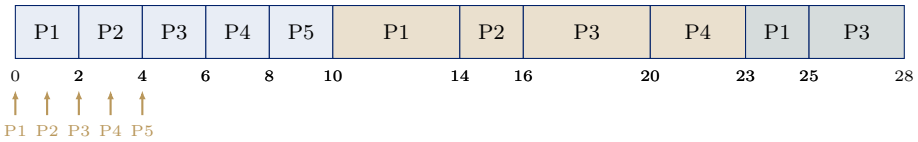


Figure 9.8: MLFQ on W (Q_0 light, Q_1 gold, Q_2 grey).

Example 9.9 (A late short arrival preempts)**.** *Same three queues;* `P1` *(arrival 0,* $\tau = 9$*),* `P2` *(1, 3),* `P3` *(2, 6),* `P4` *(8, 2). See Fig. 9.9.*

1. *0–6:* `P1`*,* `P2`*,* `P3` *each get 2 ms in* Q_0 *and are demoted;* Q_1 *= [P1 (7 left), P2 (1), P3 (4)].*
2. *6–8:* `P1` *runs in* Q_1*.*
3. $t = 8$*:* `P4` *enters* Q_0 *and preempts* `P1` *(5 left), which returns to the tail of* Q_1*, behind* `P2` *and* `P3`*.* `P4` *runs 8–10 and completes: TAT 2, WT 0 — a new short job is served at once.*
4. *10–15:* Q_1 *runs* `P2` *(10–11, done) and* `P3` *(11–15, done).*
5. *15–20:* `P1` *finishes its 5 ms (4 in* Q_1*, then demoted, 1 in* Q_2*), completing at 20.*

Completions 20, 11, 15, 10 give $TAT = 20, 10, 13, 2$ *(sum 45) and* $WT = 11, 7, 7, 0$*, so average WT* $= 25/4 = 6.25$ *ms and average* $TAT = 45/4 = 11.25$ *ms.*

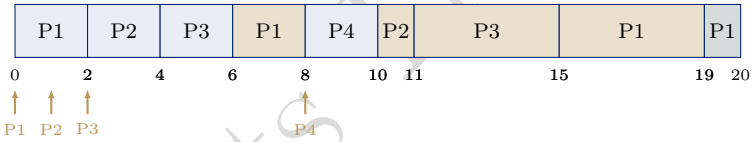


Figure 9.9: MLFQ with a late short arrival.

Gaming the scheduler

Suppose a user notices the demotion rule and makes a CPU-bound program issue a tiny I/O request just before each quantum expires. The program never "uses a full quantum", so it is never demoted and keeps Q_0's priority — it games the scheduler. The defence is to charge CPU time *cumulatively* at each level and demote once the allotment is used, however it was split. Periodic promotion (aging) protects the processes that such a program would otherwise starve.

9.6 Multiprocessor Scheduling

Reading: Silberschatz et al., Ch. 5 §5.5, p. 200 onward; Stallings, Ch. 10 §10.1, p. 453; Bovet & Cesati, Ch. 7, pp. 258–293 [1].

Everything so far assumed one CPU. With several, the scheduler must also decide *where* a process runs — and moving it is not free.

Definition (Asymmetric multiprocessing). *One master CPU runs all scheduling decisions, I/O, and kernel activity; the others execute only user code. It is simple, because only one CPU touches kernel data structures.*

Concepts That Click

Definition (Symmetric multiprocessing (SMP)). *Each CPU is self-scheduling: it examines the ready queue and picks its own next process. Scheduler data must then be protected against simultaneous access.*

Virtually all modern general-purpose OSes (Linux, Windows, macOS) are SMP, so the design question becomes: one shared ready queue, or one queue per CPU (Fig. 9.10)? With a **common queue**, any idle CPU takes the next process: balance is automatic, but every CPU contends for one lock. With **per-CPU queues** there is no contention and cache reuse is good, but the queues may drift out of balance, and the OS must actively **load-balance**.

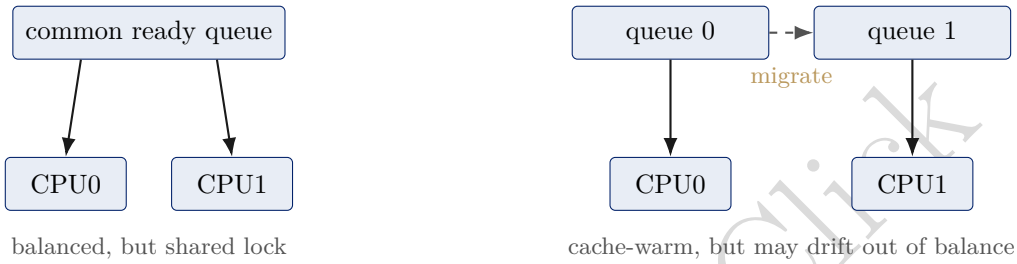


Figure 9.10: Common ready queue versus per-CPU queues.

Processor affinity

Definition (Processor affinity). *Processor affinity is the preference of a process to keep running on the CPU it last ran on, because that CPU's cache holds its working data.*

Migrating a process invalidates its cache contents on the old CPU and repopulates them on the new one, which is a real cost. Under **soft affinity** the OS tries to keep the process on the same CPU but may move it; under **hard affinity** the process restricts itself to a set of CPUs (for example with Linux `sched_setaffinity`). On NUMA machines, affinity also keeps a process near the memory it uses.

Load balancing

With per-CPU queues one CPU can sit idle while another has a long queue. In **push migration** a periodic task checks the loads and pushes processes from overloaded to less-busy CPUs; in **pull migration** an idle CPU pulls a waiting process from a busy CPU's queue. Linux uses both: per-CPU runqueues with periodic *runqueue balancing*. The two forces pull in opposite directions — load balancing moves processes, affinity resists moving them — so a good scheduler migrates only when the imbalance outweighs the cache cost.

Example 9.10 (Why balance matters)**.** *Six jobs, all ready at $t = 0$, with bursts 6, 5, 4, 3, 2, 2 ms (total 22), run on two CPUs (Fig. 9.11).*

- ***Static split*** *(J1–J3 on CPU0, J4–J6 on CPU1): CPU0 finishes at $6 + 5 + 4 = 15$, CPU1 at $3 + 2 + 2 = 7$; the work is done at 15 and CPU1 is idle for 8 ms.*
- ***Common queue*** *(next job to the first free CPU, lower-numbered CPU on ties): J1 on CPU0 (0–6), J2 on CPU1 (0–5); CPU1 frees first and takes J3 (5–9); CPU0 takes J4 (6–9); both free at 9, so J5 goes to CPU0 (9–11) and J6 to CPU1 (9–11). The work is done at 11 $= 22/2$, the best possible.*

Concepts That Click

The same work on the same CPUs: balance alone cuts the finish time from 15 to 11 ms.

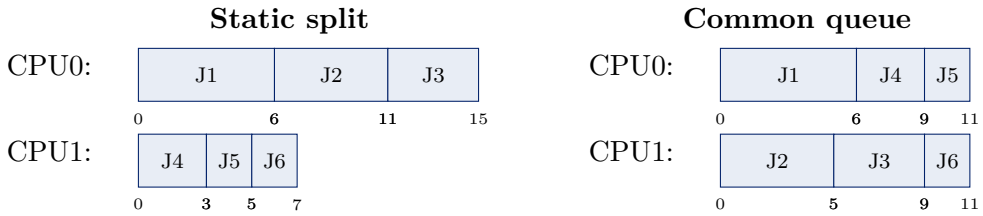


Figure 9.11: Static split versus common queue on two CPUs.

Multicore processors and memory stalls

A **memory stall** occurs when a core waits for data from memory after a cache miss; a core can spend a large share of its time stalled. Multithreaded cores therefore keep two (or more) **hardware threads**: when one stalls, the core switches to the other. The OS sees each hardware thread as a logical CPU, and the core decides which hardware thread runs from cycle to cycle. Scheduling thus happens at two levels: the OS chooses which software thread runs on each logical CPU, and the hardware chooses which hardware thread uses the core's pipeline (Silberschatz §5.5, general treatment).

9.7 Putting It Together

Policy	avg WT	avg TAT	avg RT	P5 done at	switches
FCFS	11.4	17.0	11.4	28	4
Priority (non-preemptive)	10.6	16.2	10.6	28	4
Priority (preemptive)	9.6	15.2	7.8	28	5
RR, $q = 3$	14.6	20.2	4.6	17	10
MLFQ (2, 4, FCFS)	12.8	18.4	2.0	10	10

Table 9.3: All policies on W.

Table 9.3 collects the chapter's numbers. No policy wins every column: the right choice depends on which criterion the system values. Preemptive priority minimizes average WT and TAT on this workload, RR bounds response time, and MLFQ combines RR's responsiveness with SJF-like favour for short jobs — which is why it underlies many real schedulers.

Common misconceptions

- "A higher priority number means more important." It depends on the convention; in this course, lower is higher.
- "A larger quantum always lowers waiting time." WT is not monotone in q (13.0, 14.6, 13.0 on W for $q = 2, 3, 4$).
- "RR always beats FCFS." RR improves RT, often at the cost of WT and TAT.

Concepts That Click

- "MLQ moves processes between queues." Only MLFQ moves them; MLQ assignment is permanent.
- "More CPUs always finish proportionally sooner." Only if the load is balanced (15 vs. 11 ms in the example).

Summary

Priority scheduling runs the most important ready process; preemption lets it react to arrivals, and aging prevents starvation. RR bounds response time by $(n-1)q$; the quantum trades responsiveness against context-switch overhead, and when q is at least the longest burst RR becomes FCFS. MLQ gives each fixed class its own queue and policy, while MLFQ moves processes between queues by observed behaviour. Multiprocessor scheduling adds placement: SMP, common versus per-CPU queues, affinity, and push/pull load balancing. In every calculation, draw the Gantt chart first, then compute TAT, WT, and RT per process.

Looking ahead

This chapter closes Unit I. A scheduler can stop a process at *any* timer interrupt and run another; the next lecture asks what goes wrong when two such processes share data — race conditions — and states the critical-section problem that every synchronization tool must solve. Reading: Silberschatz et al., Ch. 6 §6.1–6.2 [2]; Stallings, Ch. 5 §5.1 [3].

Self-Check

Exercises

1. For W, explain in one sentence each why non-preemptive priority could not help `P2` and why `P5`'s WT is 22 ms under every policy except RR and MLFQ.
2. A new workload: `P1` (arrival 0, $\tau = 6$, priority 2), `P2` (1, 2, 1), `P3` (2, 7, 3), `P4` (4, 3, 1). Draw the Gantt chart and give each process's WT and the average WT for (a) RR with $q = 3$, using this chapter's tie rule; (b) preemptive priority.
3. On W with $q = 3$ and $s = 0.5$ ms, what fraction of the total elapsed time is switching overhead?
4. In the late-arrival MLFQ example, change `P4`'s arrival to 10. Does `P4` still preempt anyone? What is its WT?
5. Six jobs with bursts 8, 3, 6, 2, 5, 4 ms are all ready at $t = 0$ on two CPUs. Compute the finish time for the common-queue rule (in the listed order) and for the static split (first three on CPU0, last three on CPU1).

Solutions

1. Non-preemptive priority makes a decision only when the CPU becomes free; `P1` held it from 0 to 8, so `P2` (arriving at 1) had to wait 7 ms. `P5` has the lowest priority (5) and arrives last, so FCFS and both priority forms run it last, at 26–28: WT $= 28 - 4 - 2 = 22$. RR and MLFQ give it a slice in its first round.

Concepts That Click

2. (a) RR $q = 3$: P1 0–3, P2 3–5, P3 5–8, P1 8–11, P4 11–14, P3 14–18. At $t = 3$ the queue is [P2, P3] then P1 rejoins; at 8 it is [P1, P4, P3] since P4 arrived at 4 before P3 was preempted at 8. Completions 11, 5, 18, 14; WT $= 11 - 0 - 6 = 5$, $5 - 1 - 2 = 2$, $18 - 2 - 7 = 9$, $14 - 4 - 3 = 7$; average $23/4 = 5.75$ ms.
   (b) Preemptive priority: P1 0–1, P2 1–3, P1 3–4, P4 4–7, P1 7–11, P3 11–18. At 4 `P4` (1) preempts `P1` (2). Completions 11, 3, 18, 7; WT $= 5, 0, 9, 0$; average $14/4 = 3.5$ ms.
3. 10 switches $\times$ 0.5 ms $= 5$ ms of overhead in $28 + 5 = 33$ ms elapsed, i.e. $5/33 \approx 15.2\%$.
4. At $t = 10$ `P1` has just completed its 4 ms Q_1 quantum (6–10) and is demoted, so `P4` preempts no one; it is the only Q_0 process and runs 10–12 at once: WT $= 0$.
5. Common queue: J1 CPU0 0–8, J2 CPU1 0–3, J3 CPU1 3–9, J4 CPU0 8–10, J5 CPU1 9–14, J6 CPU0 10–14: finish $14 = 28/2$. Static split: CPU0 $8 + 3 + 6 = 17$, CPU1 $2 + 5 + 4 = 11$: finish 17.

References

[1] Daniel P. Bovet and Marco Cesati. *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition, 2006.

[2] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[3] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.

Concepts That Click